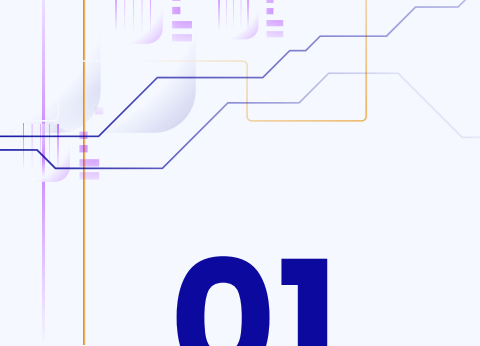




# Testowanie Aplikacji StudyHub

---

Wiktor Rzepka, Aleksandra Suder, Marcin Stępkowski



**01**

# Testy jednostkowe

---

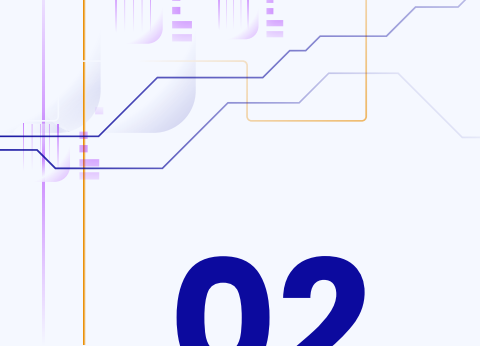


| Nr testu | Nazwa testu           | Testowana funkcja               | Co robi funkcja                         | Co sprawdza test                        | Dlaczego to ważne              |
|----------|-----------------------|---------------------------------|-----------------------------------------|-----------------------------------------|--------------------------------|
| 1        | Walidacja nazw plików | FolderView._sanitize_filename() | Usuwa niedozwolone znaki z nazw plików  | Czy znaki `< > :`                       | ? *` są usuwane                |
| 2        | Czytanie plików TXT   | read_txt()                      | Odczytuje pliki TXT w UTF-8             | Czy poprawnie czyta polskie znaki       | Poprawne wyświetlanie notatek  |
| 5        | Zmiana widoku folderu | get_notes_in_folder()           | Zwraca pliki z wybranego folderu        | Czy pokazuje tylko pliki danego folderu | Poprawna nawigacja w aplikacji |
| 8        | Konwersja TXT → JSON  | add_existing_note_file()        | Zamienia plik TXT na format .bgh (JSON) | Czy TXT konwertuje się do JSON          | Import istniejących notatek    |
| 10       | Przenoszenie plików   | move_file_to_folder()           | Przenosi plik między folderami          | Czy ulubione zostają po przeniesieniu   | Spójność danych użytkownika    |

```
1. Walidacja nazw plików
2. Czytanie plików TXT
5. Zmiana widoku folderu
8. Konwersja TXT->JSON
10. Przenoszenie plików
test_przenosi_plik_i_aktualizuje_ulubione (__main__.Test10_MoveFile.test_przenosi_plik_i_aktualizuje_ulubione) ... ok
test_usuwa_niebezpieczne_znaki (__main__.Test1_SanitizeFilename.test_usuwa_niebezpieczne_znaki) ... ok
test_czyta_polskie_znaki (__main__.Test2_ReadTxt.test_czyta_polskie_znaki) ... ok
test_widok_folderu_pokazuje_wlasciwe_pliki (__main__.Test5_FolderViewSwitch.test_widok_folderu_pokazuje_wlasciwe_pliki) ... ok
test_konwertuje_txt_na_json (__main__.Test8_AddExistingNote.test_konwertuje_txt_na_json) ... ok

-----
Ran 5 tests in 0.018s
```

Wynik uruchomienia wybranych testów jednostkowych. Wszystkie testy zakończyły się sukcesem (OK), co potwierdza poprawne działanie kluczowych funkcji aplikacji.



**02**

# **Testy komponentów**

---



# test\_kalendarz\_integracyjne.py

Cel testu: sprawdzić współpracę logiki kalendarza (calendar.py) z warstwą persystencji (calendar\_persistence.py) bez realnej bazy.

Test obejmuje:

- dodawanie wydarzeń i zapis (add\_event → create\_event)
- aktualizację (update\_event → update\_event)
- usuwanie (remove\_event → soft\_delete)
- import ICS → batch zapis (bulk\_upsert) + czyszczenie źródła (soft\_delete\_source)
- parsowanie dat i default color w CalendarPersistence.load\_events
- cleanup danych (soft/hard delete)

Kluczowe moduły:

- calendar.py – logika zdarzeń, import ICS, walidacja dat
- calendar\_persistence.py – zapis/odczyt i cleanup

# Działanie

1. Dodanie wydarzenia, zapis
2. Aktualizacja wydarzenia, zapis
3. Usunięcie wydarzenia, wykonuje soft delete
4. Import ICS: bulk upsert i czyszczenie źródła
5. Wczytanie wydarzeń parsuje ISO
6. Cleanup wywołuje Supabase

```
-----  
.TestCalendarPersistenceIntegration.test_cleanup_calls_supabase ... ok  
.TestCalendarPersistenceIntegration.test_load_events_parses_iso_strings ... ok  
.TestCalendarStoreIntegration.test_add_event_persists ... ok  
.TestCalendarStoreIntegration.test_import_ics_bulk_upsert_and_source_cleanup ... ok  
.TestCalendarStoreIntegration.test_remove_event_soft_delete ... ok  
.TestCalendarStoreIntegration.test_update_event_persists ... ok  
-----
```

Ran 6 tests in 0.010s

OK



Ln 83, Col 34, Spaces: 4

## test\_mail\_integracyjne.py

Cel testu: Sprawdzenie integracji logiki poczty w mail.py z obsługą konfiguracji, cache i klientem EmailClient – bez realnych połączeń sieciowych. Testowane pliki:

TokenStorage, ImapProvider (konfiguracja, ID wiadomości), EmailClient (przełączanie providera, foldery, send/fetch).

Testowane scenariusze:

- zapis/odczyt/usunięcie tokenów (TokenStorage)
- normalizacja konfiguracji IMAP/SMTP (porty STARTTLS) i mapowanie folderów
- walidacja błędnej konfiguracji (brak hasła → błąd)
- składanie i parsowanie ID wiadomości (folder:uid)
- przepływ klienta: wybór providera, folderu, pobranie i wysyłka



# Działanie

```
>>
1. Token storage roundtrip
2. IMAP config normalization
3. IMAP message id compose/parse
4. Email client provider flow
-----
.tests.test_mail_integracyjne.Test1-TokenStorageRoundtrip.test_save_load_clear ... ok
.tests.test_mail_integracyjne.Test2-ImapConfigNormalization.test_update_config_normalizes_ports_and_folders ... ok
.tests.test_mail_integracyjne.Test2-ImapConfigNormalization.test_update_config_requires_password ... ok
.tests.test_mail_integracyjne.Test3-ImapMessageId.test_compose_and_parse_message_id ... ok
.tests.test_mail_integracyjne.Test3-ImapMessageId.test_parse_invalid_message_id ... ok
.tests.test_mail_integracyjne.Test4-EmailClientFlow.test_fetch_and_send ... ok
.tests.test_mail_integracyjne.Test4-EmailClientFlow.test_provider_and_folder_switch ... ok
-----
Ran 7 tests in 0.039s

OK
```

## test\_notes\_integracyjne.py

Cel testu: sprawdzić współpracę kluczowych funkcji notatek (notes.py) w realistycznych scenariuszach plikowych.

Scenariusze:

- konwersja .txt → .bgh + sanitacja treści (usuwanie znaków kontrolnych)
- kopiowanie plików z dozwolonym rozszerzeniem bez konwersji
- odrzucenie niedozwolonych rozszerzeń
- sortowanie i filtrowanie listy plików w folderze
- flow ulubionych: dodanie, lista, usunięcie
- agregacja notatek ze wszystkich folderów

# Działanie

1. Konwersja TXT -> BGH z sanitacją
2. Kopiowanie pliku z dozwolonym rozszerzeniem
3. Odrzucenie niedozwolonego rozszerzenia
4. Sortowanie i filtrowanie listy plików
5. Ulubione: dodanie i usunięcie

```
-----  
.Test1_AddExistingNoteTxtConverts.test_txt_converts_to_bgh_and_sanitizes ... ok  
.Test2_AddExistingNoteCopiesAllowed.test_allowed_extension_without_reader_is_copied ... ok  
.Test3_AddExistingNoteRejectsExtension.test_rejects_disallowed_extension ... ok  
.Test4_GetNotesInFolderSorting.test_sorting_and_extension_filter ... ok  
.Test6_FavoritesFlow.test_toggle_and_list_favorites ... ok  
.  
-----
```

Ran 6 tests in 0.056s

OK



**03**

# Testy wydajności

---



**Celem testów wydajności było sprawdzenie czasu dodawania notatek oraz zachowania aplikacji przy dużej liczbie i rozmiarze plików.**

**Liczba plików:**

- 1 - 1 000 (małe obciążenie)
- 5 000 - 10 000 (średnie obciążenie)
- 50 000 - 100 000 (duże obciążenie)

**Rozmiar plików:**

- 10 KB
- 100 KB
- 1 MB
- 10 MB (kilkukrotnie)
- 50 MB (kilkukrotnie)
- 100 MB (kilkukrotnie)

# TESTY WYDAJNOSCI DODAWANIA NOTATEK

1. Testy dla roznej liczby plikow:  
- male ilosci: 1-1000  
- srednie ilosci: 5000-10000  
- duze ilosci: 50000-100000  
- bardzo duze: 500000

2. Testy dla roznych rozmiarow:  
- pojedyncze pliki: 10KB-100MB  
- wiele duzych plikow

test bardzo duze\_ilosci ( \_main \_TestWydajnosciDodawania.test bardzo duze\_ilosci) ...

TEST BARDZO DUZYCH ILOSCI (500000 plikow)

Uwaga: test moze potrwać 5-10 godzin  
Postep bedzie raportowany co 5000 plikow

Tworzenie 100,000 plików testowych...

Utworzono 5,000 plików (1.2s)  
Utworzono 10,000 plików (2.6s)  
Utworzono 15,000 plików (4.0s)  
Utworzono 20,000 plików (5.4s)  
Utworzono 25,000 plików (6.6s)  
Utworzono 30,000 plików (8.0s)  
Utworzono 35,000 plików (9.4s)  
Utworzono 40,000 plików (10.7s)  
Utworzono 45,000 plików (11.9s)  
Utworzono 50,000 plików (13.2s)  
Utworzono 55,000 plików (14.5s)  
Utworzono 60,000 plików (15.9s)  
Utworzono 65,000 plików (17.1s)  
Utworzono 70,000 plików (18.6s)

## WYNIKI TESTU

Liczba plikow: 5,000  
Calkowity czas: 6.33s  
- tworzenie plikow: 1.6s (25%)  
- przetwarzanie: 4.7s (75%)  
Sredni czas na plik: 0.000949s  
Przepustowosc: 1,054 plikow/s

Tworzenie 10,000 plików testowych...

Utworzono 5,000 plików (1.5s)  
Utworzono 10,000 plików (3.2s)  
Zakończono tworzenie w 3.2s

Przetwarzanie plików...

5,000/10,000 (50%) - uplynelo 4s, pozostalo ~4s, 1226 plikow/s  
10,000/10,000 (100%) - uplynelo 8s, pozostalo ~0s, 1215 plikow/s  
Zakonczono przetwarzanie w 8.2s (1215 plikow/s)

+ Program PowerShell dla developerów

Utworzono 90,000 plików (24.8s)  
Utworzono 95,000 plików (26.4s)  
Utworzono 100,000 plików (28.0s)  
Zakończono tworzenie w 28.0s

Przetwarzanie plików...

5,000/100,000 (5%) - uplynelo 8s, pozostalo ~156s, 611 plikow/s  
10,000/100,000 (10%) - uplynelo 17s, pozostalo ~150s, 599 plikow/s  
15,000/100,000 (15%) - uplynelo 25s, pozostalo ~141s, 604 plikow/s  
20,000/100,000 (20%) - uplynelo 33s, pozostalo ~134s, 598 plikow/s  
25,000/100,000 (25%) - uplynelo 42s, pozostalo ~125s, 600 plikow/s  
30,000/100,000 (30%) - uplynelo 53s, pozostalo ~125s, 562 plikow/s  
35,000/100,000 (35%) - uplynelo 66s, pozostalo ~122s, 532 plikow/s  
40,000/100,000 (40%) - uplynelo 77s, pozostalo ~115s, 520 plikow/s  
45,000/100,000 (45%) - uplynelo 85s, pozostalo ~104s, 531 plikow/s  
50,000/100,000 (50%) - uplynelo 93s, pozostalo ~93s, 539 plikow/s  
55,000/100,000 (55%) - uplynelo 101s, pozostalo ~83s, 543 plikow/s  
60,000/100,000 (60%) - uplynelo 109s, pozostalo ~73s, 549 plikow/s  
65,000/100,000 (65%) - uplynelo 120s, pozostalo ~65s, 540 plikow/s  
70,000/100,000 (70%) - uplynelo 128s, pozostalo ~55s, 547 plikow/s  
75,000/100,000 (75%) - uplynelo 136s, pozostalo ~45s, 552 plikow/s  
80,000/100,000 (80%) - uplynelo 144s, pozostalo ~36s, 555 plikow/s  
85,000/100,000 (85%) - uplynelo 152s, pozostalo ~27s, 559 plikow/s  
90,000/100,000 (90%) - uplynelo 161s, pozostalo ~18s, 559 plikow/s  
95,000/100,000 (95%) - uplynelo 170s, pozostalo ~9s, 560 plikow/s  
100,000/100,000 (100%) - uplynelo 178s, pozostalo ~0s, 562 plikow/s  
Zakonczono przetwarzanie w 177.8s (562 plikow/s)

## WYNIKI TESTU

Liczba plikow: 100,000  
Calkowity czas: 205.8s (3.4min)  
- tworzenie plikow: 28.0s (14%)  
- przetwarzanie: 177.8s (86%)  
Sredni czas na plik: 0.001778s  
Przepustowosc: 562 plikow/s

Przetwarzanie plików...

5,000/50,000 (10%) - uplynelo 4s, pozostalo ~38s, 1186 plikow/s  
10,000/50,000 (20%) - uplynelo 8s, pozostalo ~34s, 1194 plikow/s  
15,000/50,000 (30%) - uplynelo 13s, pozostalo ~30s, 1182 plikow/s  
20,000/50,000 (40%) - uplynelo 17s, pozostalo ~26s, 1170 plikow/s  
25,000/50,000 (50%) - uplynelo 21s, pozostalo ~21s, 1175 plikow/s  
30,000/50,000 (60%) - uplynelo 26s, pozostalo ~17s, 1170 plikow/s  
35,000/50,000 (70%) - uplynelo 30s, pozostalo ~13s, 1174 plikow/s  
40,000/50,000 (80%) - uplynelo 34s, pozostalo ~9s, 1167 plikow/s  
45,000/50,000 (90%) - uplynelo 39s, pozostalo ~4s, 1163 plikow/s  
50,000/50,000 (100%) - uplynelo 43s, pozostalo ~0s, 1167 plikow/s  
Zakonczono przetwarzanie w 42.9s (1167 plikow/s)

## WYNIKI TESTU

Liczba plikow: 50,000  
Calkowity czas: 59.77s  
- tworzenie plikow: 16.9s (28%)  
- przetwarzanie: 42.9s (72%)  
Sredni czas na plik: 0.000857s  
Przepustowosc: 1,167 plikow/s

## WYNIKI TESTU

Liczba plikow: 10  
Calkowity czas: 0.01s  
Sredni czas na plik: 0.000992s  
Przepustowosc: 1,008 plikow/s

## WYNIKI TESTU

Liczba plikow: 50  
Calkowity czas: 0.04s  
Sredni czas na plik: 0.000868s  
Przepustowosc: 1,151 plikow/s

## WYNIKI TESTU

Liczba plikow: 100  
Calkowity czas: 0.09s  
Sredni czas na plik: 0.000937s  
Przepustowosc: 1,068 plikow/s



# TEST ROZNYCH ROZMIAROW PLIKOW

```
Tworzenie 1 plikow...
Przetwarzanie...
10KB: 0.0015s (6.47 MB/s)
Tworzenie 1 plikow...
Przetwarzanie...
100KB: 0.0022s (44.64 MB/s)
Tworzenie 1 plikow...
Przetwarzanie...
1.0MB: 0.0120s (83.35 MB/s)
Tworzenie 1 plikow...
Przetwarzanie...
10.0MB: 0.1064s (93.96 MB/s)
Tworzenie 1 plikow...
Przetwarzanie...
50.0MB: 0.1586s (315.30 MB/s)
Tworzenie 1 plikow...
Przetwarzanie...
100.0MB: 0.2088s (479.00 MB/s)
```

ok

test\_wiele\_duzych\_plikow (\_\_main\_\_.TestWydajnosciDuzzePliki.test\_wiele\_duzych\_plikow) ...

```
10 plikow po 1.0MB:
Tworzenie 10 plikow...
Przetwarzanie...
  Calkowity czas: 0.12s, srednia: 0.012s/plik
```

```
5 plikow po 10.0MB:
Tworzenie 5 plikow...
Przetwarzanie...
  Calkowity czas: 0.55s, srednia: 0.111s/plik
```

```
2 plikow po 50.0MB:
Tworzenie 2 plikow...
Przetwarzanie...
  Calkowity czas: 0.29s, srednia: 0.145s/plik
```

ok

-----  
Ran 6 tests in 605.860s

# 04

## Fuzz testing (Fuzzing)

---



# test\_calendar\_fuzz.py

Cel testu: fuzzing logiki kalendarza: losowe dane wejściowe mają wykryć awarie i błędy w przetwarzaniu dat oraz importu ICS. Testowane pliki/kod: calendar.py. (CalendarStore.add\_event, CalendarStore.import\_ics, \_as\_datetime).

## Scenariusze:

### TestCalendarFuzzAdd

Test test\_add\_event\_random generuje losowe tytuły i czasy, dodaje wydarzenia, sprawdza brak wyjątków dla poprawnego zakresu danych wejściowych. Używa funkcji CalendarStore.add\_event.

### TestCalendarFuzzImport

Test test\_import\_ics\_random\_payloads generuje losowe pliki ICS (pojedynczy event) i importuje je, sprawdza brak awarii przy poprawnie sformatowanych, ale losowych danych. Wykorzystuje funkcję CalendarStore.import\_ics

### TestCalendarFuzzAsDatetime

Test test\_as\_datetime\_random przekazuje losowo datetime albo date do \_as\_datetime i sprawdza zawsze zwracany jest poprawny obiekt datetime. Funkcje użyte \_as\_datetime.

# Działanie

Ran 3 tests in 0.022s

FAILED (errors=1)

| studyhub calendar fuzz (test) |

|                |                    |  |
|----------------|--------------------|--|
| process timing | run time: 0h 0m 0s |  |
|----------------|--------------------|--|

|                 |                     |  |
|-----------------|---------------------|--|
| overall results | total cases : 401   |  |
|                 | total crashes: 1    |  |
|                 | last crash : import |  |

|                |             |  |
|----------------|-------------|--|
| stage progress | add : 200   |  |
|                | import : 1  |  |
|                | as_dt : 200 |  |

# test\_notes\_fuzz.py

Cel testu to fuzzing modułu notatek, sprawdzenie odporności na losowe dane wejściowe w kluczowych funkcjach. Testowane pliki/kod notes.py

## Scenariusze

### TestNotesFuzzSanitize

Test `test_sanitize_text_random_inputs` generuje losowe ciągi (0–1024 znaki ASCII) i przepuszcza przez `_sanitize_text`, sprawdza czy długość nie przekracza `MAX_CONTENT_LENGTH` oraz brak znaków kontrolnych w wyniku. Funkcje użyte: `_sanitize_text`.

### TestNotesFuzzAddExisting

Test `test_add_existing_note_file_random_txt` tworzy losowe pliki `.txt` i dodaje je jako notatki i sprawdza czy wynikowy plik istnieje, następuje konwersja do `.bgh`. Funkcje użyte to `add_existing_note_file`

Test `test_add_existing_note_file_invalid_folder` próbuje dodać plik do nieprawidłowego folderu (`../bad`). Sprawdza `ValueError` używając funkcji `add_existing_note_file`

### TestNotesFuzzGetNotes

Test: `test_get_notes_in_folder_random_names` losuje nazwy folderów i wywołuje `get_notes_in_folder`, sprawdza czy funkcja zawsze zwraca listę i nie rzuca wyjątków. Używa funkcji `get_notes_in_folder`

# Działanie

....

Ran 4 tests in 1.219s

OK

|                            |                    |
|----------------------------|--------------------|
| studyhub notes fuzz (test) |                    |
| process timing             | run time: 0h 0m 1s |
| overall results            | total cases : 350  |
|                            | total crashes: 0   |
|                            | last crash : none  |
| stage progress             | sanitize: 200      |
|                            | add : 50           |
|                            | get : 100          |

# test\_validation\_fuzz.py

Celem testu jest fuzzing walidatorów wejścia – losowe dane mają wykryć błędy i wyjątki w funkcjach walidujących użytkownika, hasło, e-mail oraz nagłówki/załączniki. Testowane pliki/kod: validation.py

## Scenariusze:

**TestValidationFuzzUsername**, test\_validate\_username\_random generuje losowe ciągi (0–40 znaków) i sprawdza, że funkcja zwraca bool. Dodatkowo weryfikuje przypadki graniczne (za krótkie, za długie, wielkie litery). Używa funkcji validate\_username.

**TestValidationFuzzPassword**, test\_validate\_password\_random generuje losowe hasła i sprawdza, że funkcja zwraca bool. Dodatkowo testuje granice długości (min/max). Używa funkcji validate\_password.

**TestValidationFuzzEmails**, test\_validate\_emails\_random generuje losowe ciągi, oczekując listy e-maili lub ValueError. Dodatkowo sprawdza poprawny e-mail i błędny format. Używa funkcji validate\_emails.

**TestValidationFuzzSanitizeHeader**, test\_sanitize\_header\_random generuje losowe nagłówki i sprawdza brak \n/\r oraz limit 255 znaków. Używa funkcji sanitize\_header.

**TestValidationFuzzAttachmentName**, test\_validate\_attachment\_name\_random generuje losowe nazwy plików i sprawdza brak awarii. Dodatkowo wymusza błąd dla .exe. Używa funkcji validate\_attachment\_name.

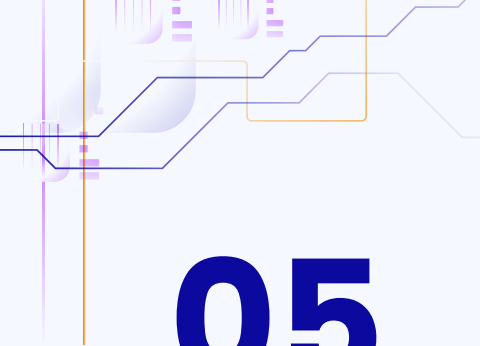
# Działanie

.....

Ran 5 tests in 0.043s

OK

|                                 |                                                                                   |
|---------------------------------|-----------------------------------------------------------------------------------|
| studyhub validation fuzz (test) |                                                                                   |
| process timing                  | run time: 0h 0m 0s                                                                |
| overall results                 | total cases : 1200<br>total crashes: 0<br>last crash : none                       |
| stage progress                  | username: 300<br>password: 300<br>emails : 200<br>header : 200<br>attachment: 200 |

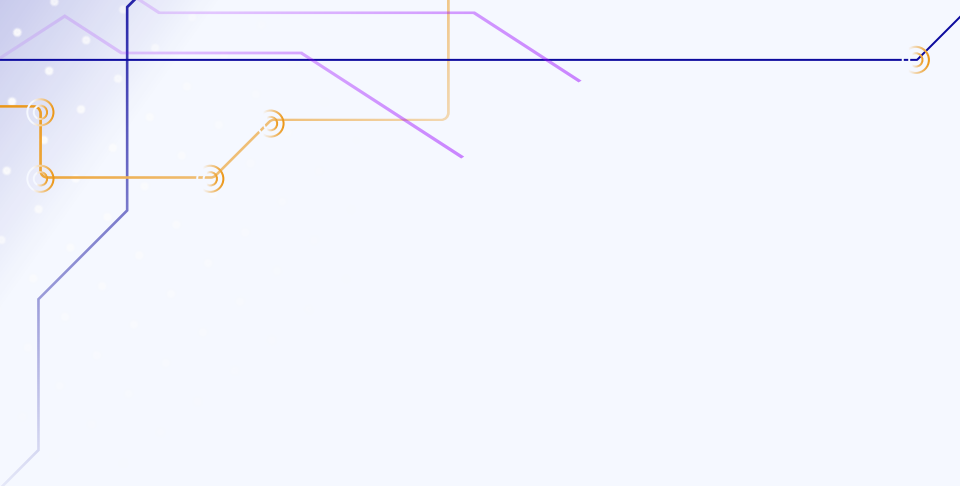


**05**

# **Testy akceptacyjne UAT**

---



Abstract circuit-like lines in blue, orange, and purple, with small circular nodes, extending from the top-left corner.

# KONIEC

Wiktor Rzepka, Aleksandra Suder, Marcin Stępkowski

Abstract circuit-like lines in blue, orange, and purple, with small circular nodes, extending from the bottom-right corner.